מספר תעודת זהות
יש לרשום את כל תשע הספרות

יש להדביק כאן את מדבקת הנבחן

האוניברסיטה הפתוחה

כ"ח בתמוז תשפ"ג

מס' שאלון - 524

17
ביולי 2023

סמסטר 2023ב

מס' מועד 87

4 / 20905

שאלון בחינת גמר

20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה **7** עמודים

מבנה הבחינה:

בבחינה 3 שאלות.
משך זמן הבחינה 3 שעות.
עליכם לענות על כל השאלות.
משקל השאלות מפורט בגוף השאלון.

שימו לב:
את תשובותיכם רשמו אך ורק במחברת הבחינה.
כתבו בכתב יד ברור ומסודר.

חומר עזר המותר בשימוש:
ספר הקורס "Essentials of Programming Languages".
מדריך הלמידה "שפות תכנות".

חומר עזר:
ספר הקורס: essentials of programming languages
מדריך הלמידה. מותרות הערות בכתב יד, ע"ג הספרים.
אין להכניס חומר מודפס, דיגיטלי או כל חומר אחר מכל סוג שהוא.

בהצלחה !!!

אינכם חייבים
להחזיר את השאלון לאוניברסיטה הפתוחה

שאלה 1 (35 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 81-74. שפה זו מאפשרת הגדרה והפעלה של פרוצדורות לא רקורסיביות.
בשאלה זו, נעשה שימוש במפרש של שפת "וישגור" המממש סביבה באמצעות מבני נתונים.

נרצה להוסיף לשפת "וישגור" ביטוי חדש, המאפשר גישה למשתנה גלובלי במידה וקיים.

למשל, אם בזמן הגדרת פרוצדורה קיים משתנה בשם a וגם לפרוצדורה יש פרמטר בשם a, כאשר הפרוצדורה מופעלת, ומתוך הגוף שלה ניגשים למשתנה a, הגישה נעשית אל הפרמטר של הפרוצדורה ולא אל ה-a הגלובלי יותר (הפרמטר של הפרוצדורה מסתיר משתנים אחרים באותו שם).

בביטוי החדש נרצה לאפשר למתכנת לגשת אל המשתנה הגלובלי (במידה וקיים) ולעקוף את הלוקלי באותו שם.

לשם כך, נוסיף לשפה את הביטוי החדש הבא:

Expression ::= :: ***Identifier***

```
global-exp (id)
```

הסבר:

הביטוי מורכב מ- :: ולאחריהם שם של משתנה.

חישוב ביטוי זה יאפשר גישה למשתנה ששמו מיוצג ע"י id , ושאינו האחרון המוכר למפרש, אלא הקודם לו.

במידה ולא קיים מופע כזה תודפס שגיאה.

<u>הערה</u>: ביטוי חדש זה הוא בנוסף לביטוי var-exp הקיים בשפה הממשיך לפעול כרגיל.

להלן מספר דוגמאות להמחשת השימוש:

דוגמה 1:

```
> (run "let a=100
        in
         let p = proc (a)  -(a,::a)
         in
          (p 500)")
(num-val 400)
>
```

בדוגמה זו, מוגדר משתנה בשם a עם ערך 100, לאחר מכן מוגדרת פרוצדורה p עם פרמטר באותו שם a. לבסוף מופעלת הפרוצדורה עם ארגומנט שערכו 500. גוף הפרוצדורה מבצע חיסור בין a הפרמטר (שקיבל ערך 500) לבין a הגלובלי יותר, שערכו 100 , ועל כן תוצאת חישוב התוכנית היא 400.

דוגמה 2:

```
> (run "let a=100
        in
         let p = proc (b)  -(b,::a)
         in
          (p 500)")
environments.scm:55:8: apply-env: No binding for a
>
```

בדוגמה זו, לפרוצדורה p יש פרמטר בשם b, ומתוך גוף הפרוצדורה יש ניסיון לגשת ל-a שהוא גלובלי מעבר ל-a אותו מכירה הפרוצדורה. הפרוצדורה מכירה את a שערכו 100, אך מעליו אין גלובלי יותר, ולכן הודפסה שגיאה.

דוגמה 3:

```
> (run "let a=100
        in
          let a=700
        in
         let p = proc (b)  -(a,::a)
         in
          (p 500)")
(num-val 600)
>
```

בדוגמה זו, מוגדר משתנה a עם ערך 100, לאחר מכן, מוגדר אחד נוסף עם ערך 700, לאחר מכן מוגדרת פרוצדורה עם פרמטר בשם b , המחזירה את תוצאת החיסור בין a שמוכר לה (700) לבין a שהוא גלובלי יותר (100), ולכן הפעלת הפרוצדורה החזירה 600.

דוגמה 4:

```
> (run "let a=100
        in
          let a=700
        in
         let p = proc (b)  -(a,::c)
         in
          (p 500)")
environments.scm:55:8: apply-env: No binding for c
>
```

בדוגמה זו, ישנה גישה למשתנה גלובלי c שאינו מוכר כלל ולכן הוחזרה שגיאה.

ממשו והטמיעו את השינויים הדרושים כפי שהוגדרו והודגמו בתוך שפת **"יישגור"** (**שפת PROC**). בתשובתכם, הקפידו להסביר **היכן בדיוק** נדרשים שינויים ותוספות בקבצי המפרש, **ומהם** השינויים והתוספות בקוד המפרש. ייתכנו מספר דרכי מימוש שונים.

הנחיות כלליות לפתרון:

1. הקפידו על כתב ברור, ועל קוד מבני ומסודר.
2. הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
3. הפתרון אמור לשמור על הגדרות השפה המקוריות (פרט למקרים בהם נדרש שינוי כזה במפורש בשאלה)
4. על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
5. בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
6. ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב- terminals, non-terminals ופעולות של סגור ($\{\ \}^*$ או $\{\ \}^+$)

שאלה 2 (35 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "**וירמוז**"(IMPLICIT-REFS).

תזכורת: ביטוי let מאפשר להוסיף לסביבה משתנה זמני עם ערך עבורו. ואז לחשב את גוף ה-let באמצעות סביבה זמנית זו.
בשאלה זו אין שינוי בתחביר השפה, אלא רק באופן החישוב של ביטויים קיימים בשפה.

נרצה לשנות את האופן בו ביטוי let מחושב, ולאפשר במסגרתו חישוב עצל (lazy) של קריאות לפרוצדורה.
כאשר מוגדר בביטוי let משתנה זמני המקבל כערך תוצאה של הפעלת פרוצדורה, נרצה שהפרוצדורה לא תופעל בשלב זה (כלומר ערכו של המשתנה הזמני לא יחושב כרגע). בגוף ה- let בכל פעם שנדרש ערכו של המשתנה רק אז יחושב ערכו (עפ"י הסביבה שהיתה זמינה בעת הגדרת המשתנה).

התנהגות זו צריכה להיות תקפה רק בביטוי let שבו מוגדר משתנה זמני שערכו הוא תוצאה של ביטוי הפעלת פרוצדורה, כל מקרה אחר, ה-let יחושב כרגיל.

להלן דוגמה להמחשת ההתנהגות הרצויה:

```
> (run "
      let c=300
      in
        let a=700
        in
         let p= proc (b)
                begin
                  set a=b;
                  if zero? (-(b,40))
                  then 2000
                  else 5000
                end
         in
          let y= (p c)
          in
           begin
                -(y,50);
                set c= 8000;
                -(y,a)
           end")
(num-val -3000)
>
```

הסבר הדוגמה בעמוד הבא:

הסבר:

בדוגמה זו, מוגדרים המשתנים: c=300, a=700 ו-p פרוצדורה. לאחר מכן, ישנו ביטוי let (הרביעי) המגדיר משתנה בשם y שערכו הוא תוצאת הפעלת הפרוצדורה p עם ארגומנט הנתון ע"י ערכו של c. על פי ההתנהגות הרצויה, הפרוצדורה לא מופעלת בשלב הגדרת המשתנה הזמני y, אלא בכל פעם שיש שימוש בערכו של y בפועל. בגוף ה-let מבוצעים מספר ביטויים, הביטוי הראשון הוא חיסור בין ערכו של y לבין 50, לכן בשלב זה מופעלת הפרוצדורה p עם ארגומנט 300 (ערכו של c) על מנת שנדע את ערכו של y לצורך ביצוע החיסור. כתוצאה מהפעלת p, ערכו של a משתנה ל-300. לאחר ביצוע פעולת החיסור, מבצעים שינוי ערך למשתנה c ל- 8000. וכעת מבוצע הביטוי האחרון בגוף ה- let שתוצאתו היא תוצאת כל התוכנית. ביטוי זה מבצע חיסור בין y לבין a. ושוב מחושב ערכו של y מחדש, ולכן הפרוצדורה p מופעלת שוב, עם ארגמונט 8000 (c השתנה), במהלך ההפעלה, ערכו של a משתנה ל-8000 והפרוצדורה מחזירה ערך של 5000. לאחר חישוב y במסגרת פעולת החיסור האחרונה, מחושב ערכו של a והוא 8000 ולבסוף מבוצעת פעולת החיסור 8000-5000 המחזירה תשובה של 3000-

ממשו והטמיעו את השינויים הדרושים בתוך שפת **"וירמוז"** (**שפת IMPLICIT-REFS**).
בתשובתכם, הקפידו להסביר **היכן בדיוק** נדרשים שינויים ותוספות בקבצי המפרש, **ומהם** השינויים והתוספות בקוד המפרש.

הנחיות כלליות לפתרון:

1. הקפידו על כתב ברור, ועל קוד מבני ומסודר.
2. הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
3. הפתרון אמור לשמור על הגדרות השפה המקוריות. (כגון אופן ניהול הסביבה וסוגי הערכים בה, אופן ניהול ההחסן והערכים שניתן לשמור בו, וכדומה)
4. על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
5. בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
6. ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב- terminals, non-terminals ופעולות של סגור ($\{\ \}^*$ או $\{\ \}^+$)

המשך הבחינה בעמוד הבא

שאלה 3 (30 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

(proc (x: ?) x proc (y: ?) -(y,y))

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים).לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (15 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב- substitution ו- unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

בהצלחה!